DIÁRIO DE PROMPTS

Informações importantes

Os testes foram realizados usando o ChatGPT como desenvolvedor de código, em uma conta gratuita e removendo a referência de memórias salvas para não lembrar de detalhes de chats e influenciar no resultado esperado.

ChatGPT

Prompt (Versão 1.0) – 17 de novembro de 2025

"De acordo com a documentação enviada sobre esta aplicação que pretendo implementar, forneça o código passo-a-passo para ela a fim de que eu possa ir transportando para o meu VSCode (me oriente inclusive sobre como organizar o ambiente para este projeto). Adote uma abordagem incremental, fornecendo o código por etapas na ordem que julgar melhor, avançando para a próxima etapa apenas quando eu disser que concluí a atual. Quando todas as etapas estiverem concluídas, me informe sobre isto indicando que o projeto está pronto."

1ª ETAPA

OBSERVAÇÕES INICIAIS

Nesta primeira versão do prompt, o ChatGPT se comportou como o esperado, porém, entregou um arquivo geral em markdown com os primeiros passos para a criação do projeto economizApp. Ensinou como preparar o ambiente (instalação do VSCode, Flutter), configuração de emuladores e criação de um novo projeto.

PROBLEMAS ENCONTRADOS

Seguindo os passos fornecidos me deparo com um problema nas versão das dependencias utilizadas no pubspec.yaml que são bastante antigas e podem comprometer o funcionamento do projeto no futuro. Isso é bastante perceptível usando IA por que muitas vezes os conteúdos em que são treinadas são antigos, então, pode ser que utilizamos códigos que já estão depreciados. Apesar que o

ChatGPT notificou que as versões são de exemplo, e que devemos aceitar as versão que o flutter pub get recomendar, sendo assim, fiz o que foi pedido pelo GPT.

Na parte de estrutura do projeto, foi indicado uma arquitetura básica de camadas, separando a camada de dados chamada de “services”, e “models” que não deixa claro se é da camada de dominio ou de dados, mas como não existe usecases, acredito ser da camada de dados. Por isso, a estrutura do projeto é simples e não segue uma arquitetura que otimiza a manutenibilidade e testes.

Estrutura de pastas sugerida:

```
📦 economiz_app
├ 📂 android
├ 📂 ios
├ 📂 lib
│ ├ 📂 models
│ │ ├ 📄 market.dart
│ │ └ 📄 product.dart
│ ├ 📂 providers
│ │ └ 📄 app_state.dart
│ ├ 📂 services
│ │ ├ 📄 auth_service.dart
│ │ ├ 📄 firestore_service.dart
│ │ └ 📄 storage_service.dart
│ ├ 📂 screens
│ │ ├ 📄 splash_screen.dart
│ │ ├ 📄 market_list_screen.dart
│ │ ├ 📄 market_profile_screen.dart
│ │ ├ 📄 product_detail_screen.dart
│ │ ├ 📄 admin_login_screen.dart
│ │ ├ 📄 admin_register_screen.dart
│ │ └ 📄 admin_panel_screen.dart
│ └ 📂 widgets
│ │ ├ 📄 market_card.dart
│ │ └ 📄 product_card.dart
│ ├ 📄 main.dart
└ 📄 pubspec.yaml
```

2ª ETAPA

OBSERVAÇÕES INICIAIS

Para a etapa 2 o ChatGPT trabalhou com a configuração do Firebase mais detalhada, conversão de Timestamp (um tipo para trabalhar com datas no CloudFirestore), cadastro de mercado e uploud de logo.

PROBLEMAS ENCONTRADOS

Aconteceu erros relacionados ao tipo de arquivo na configuração do Firebase no projeto, mais especificamente no build.gradle onde existem 2 tipos de sintaxe:

A sintaxe do Groovy e a sintaxe do Kotlin DSL. No projeto utilizamos o Kotlin DSL mas no ChatGPT utilizou o Groovy, e foi preciso descobrir esse erro. Para a continuação da configuração do Firebase foi necessário usar o Kotlin DSL

Tentativa de cadastro de mercado e um erro ocorreu (FirebaseException ([firebase_storage/object-not-found] No object exists at the desired reference.). Esse Erro está relacionado ao caminho usado ao obter a URL de download não existe no Storage do Firebase e isso está ligado as regras no Firebase Storage que devem permitir leitura e escrita, neste caso, tive que intervir e pedir ajuda para solucionar o problema. A solução dada é mudar o plano de utilização do Firebase de gratuito para pago (somente se exceder os limites do plano gratuito), que assim tenho acesso as regras do Storage. Outra opção é utilizar o Supabase para resolver este problema.

Percebi tambem que não foram feitos tratamentos de exceção, tentei criar 2 supermercados com o mesmo email e foi gerado erro que foi capturado pelo Flutter, mas não foi trabalhado de forma visual para o usuário.

No arquivo storage_service.dart os seguintes codigos foram acusados de erro (FirebaseException ([firebase_storage/object-not-found] No object exists at the desired reference.):

```
final uploadTask = await ref.putFile(file);
final url = await ref.getDownloadURL();
```

as correções recomendadas foram as seguintes:

```
final uploadTask = ref.putFile(file);
final snapshot = await uploadTask.whenComplete(() => null);
final url = await snapshot.ref.getDownloadURL();
```

Mas a fonte desse erro já foi descrito como sendo a ausencia de permissão de leitura e escrita no Storage do Firebase.

CONCLUSÃO

O ChatGPT não seguiu a proposta de projeto descrita na documentação, na etapa RNF006 (Armazenamento) em que os dados armazenados deveriam estar

em serviços gratuitos. A escolha da IA foi de utilizar o Firebase, porém, o serviço de Storage deste produto possui planos de uso pago caso o cliente exceda o limite gratuito. A escolha ideal para essa situação deveria ser o Supabase Storage que é um serviço livre de pagamentos e não utilizamos o cartão de crédito para uma possível cobrança. Houve intervenção durante o processo de desenvolvimento do projeto, principalmente para tentar resolver os problemas descritos.

Prompt (Versão 2.0) – 18 de novembro de 2025

Foi utilizado o prompt (1.0) mas com alteração no RNF 006 que é sobre o armazenamento de dados da aplicação. Antes no documento estava da seguinte forma: As fotos (logos, produtos) devem ser armazenadas em um serviço de *storage* gratuito e não diretamente no banco de dados. Dessa forma deixamos a IA escolher o serviço de armazenamento, porém, isso fez com que utilizasse o Firebase em que tivemos problemas que serão descritos, sendo assim, houve modificação e ficou da seguinte maneira: As fotos (logos, produtos) devem ser armazenadas em um serviço de *storage* gratuito (Supabase) e não diretamente no banco de dados. Dessa forma especificamos que o storage seria o Supabase.

ETAPAS DE DESENVOLVIMENTO

1ª ETAPA

Nesta etapa foram desenvolvidos os seguintes itens:

- Comando para criar o projeto Flutter.
- Estrutura de pastas sugerida.
- Dependências iniciais e .env para chaves do Supabase.
- Código mínimo main.dart para iniciar a navegação entre os dois modos.

A IA não deixou claro que o arquivo .env deveria estar listado no pubspec.yaml e na execução do código o pacote flutter_dotenv não conseguiu carregar dando uma exceção FileNotFoundError.

OBSERVAÇÕES DA IA

OBSERVAÇÕES DE SEGURANÇA E ARQUITETURA

• Recomendo fortemente não guardar senhas em texto simples no banco. Mesmo que a documentação diga "senhas devem ser armazenadas no banco de dados", implemente hashing (bcrypt) ou utilize a autenticação nativa do Supabase (Recommended).

• Use Supabase Storage para as imagens (RNF006).

• Gerencie a visibilidade de promoções (RNF007) usando filtros por start_at e end_at no backend (query) e no lado do cliente.

2ª ETAPA

Nesta etapa foram desenvolvidos os seguintes itens:

• Criar o projeto no Supabase
• Criar tabelas
• Configurar policies (RLS)
• Configurar o Supabase no app
• Criar telas AdminLogin e AdminRegister
• Implementar autenticação e salvar dados do mercado no banco

Na configuração de politicas foi necessário desativá-la por que estava tendo muito bloqueio na criação de um mercado, mas é importante desativar por questão de segurança, caso criem contas com e-mails iguais. O e-mail deve ser válido para receber uma confirmação e criar a conta de fato.

3ª ETAPA

Nesta etapa foram desenvolvidos os seguintes itens:

• Configuração do bucket no Supabase
• Upload de imagens (logo do mercado e fotos de produtos)
• Atualização do perfil do mercado com logo
• reparação para o CRUD de produtos

Encontrei algumas dificuldades na configuração do Bucket para armazenar imagem que não foram esclarecidas no processo da etapa em questão, foi preciso perguntar e solucionar problemas de acesso a imagem do mercado cadastrado.

4ª ETAPA

Nesta etapa foram desenvolvidos os seguintes itens:

- Modelo Promotion
- Serviço de banco (promotion_service.dart)
- Tela de listagem de promoções
- Tela de criação/edição
- Upload de imagem do produto
- Filtragem por data de validade (promoções ativas)

Foi preciso ser criado uma tela para editar os dados do mercado que não estava na listagem, isso foi preciso por que a próxima etapa (5ª etapa) é focado 100% no usuário comum (pessoa fisica). Tive problemas com a falta de clareza com a criação de banco dos produtos e também de politicas dos produtos. A IA também fez confusão com a filtragem por data de validade para promoções ativas, sugerindo que produtos fora do prazo de promoção não aparecesse para o mercado gerenciar, o que é um erro já que o mercado precisa remover ou atualizar seus produtos e datas.

5ª ETAPA

Este módulo terá 5 telas principais:

5.1 – Tela Inicial do Cliente (Lista de Mercados)

Mostra todos os mercados cadastrados com foto e dados básicos.

5.2 – Tela de Promoções do Mercado

Ao clicar em um mercado, abre a lista de promoções ativas daquele mercado.

5.3 – Tela Detalhe da Promoção

Exibe foto grande, preço, descrição e botão para salvar como favorito (opcional).

5.4 – Tela de Busca por Produtos (opcional por agora)

Busca global filtrando por “pão”, “frango”, “leite”, etc.

5.5 – Tela de Favoritos (opcional, mas fácil)

Para salvar promoções que o usuário gosta.

Nesta etapa começou a fugir do escopo do documento do projeto, indicou o uso de recurso de favoritar, os usuários não criam conta então não é possível criar o

recurso de favoritos. Indiquei que seguisse o fluxo que desejasse e a ideia foi criar uma tela home central para o usuário com bottomMenu indicando a criação do recurso de favoritos que foge do escopo do projeto. Então foi questionado se lembrava do documento com os requisitos funcionais e não funcionais e foi necessário reenviar o documento para verificar novamente.

Foi pedido para que fizesse uma verificação completa do documento e informar o que falta para a conclusão do projeto, a resposta foi a seguinte:

✅ 🔍 Análise Geral – O que o projeto realmente pede
Com base na documentação, o EconomizApp NÃO possui:
❌ Favoritos
❌ Carrinho
❌ Avaliações
❌ Login de usuário comum
❌ Recomendação
❌ Filtros avançados
❌ Sistema de níveis ou cupons
E também NÃO existe workflow para:

- Usuário salvar mercados
- Usuário salvar produtos
- Usuário criar conta

➡️ Portanto, realmente favoritos não faz parte do escopo.
Você estava totalmente correto em levantar essa dúvida.

✅ 💡 O sistema é essencialmente um “catálogo de promoções” unidirecional
Fluxo correto segundo a documentação:
Cliente Administrador (Mercado)
✔ Cadastra conta
✔ Faz login
✔ Configura o perfil do mercado

✔ Cria promoções

✔ Edita promoções

✔ Exclui promoções

✔ Edita dados do mercado (logo, telefone, endereço, etc.)

Esse módulo já desenvolvemos praticamente 100%.

Ainda faltam pequenos ajustes que vou listar em seguida.

Cliente Comum (Usuário) – SEM LOGIN

✔ Ver lista de mercados

✔ Pesquisar mercados

✔ Ver perfil do mercado

✔ Ver promoções ativas

✔ Ver detalhes da promoção

Esse módulo está quase finalizado.

Faltam só alguns detalhes que vou listar abaixo.

Super Usuário (Admin Geral da Aplicação)

- 🔸 Apenas mencionado na documentação
- 🔸 Não tem requisitos detalhados
- 🔸 Pode ser adicionado no final como painel básico

📌 Checklist Completo – O que já temos / O que falta

Vou marcar tudo com:

🟩 Pronto

🟨 Parcial

🟥 Falta implementar

RF-A — Módulo Administrador (Mercado)

🟩 RF-A001 Cadastro de Mercado

✔ Cadastro

✔ Logo

✔ Endereço

✔ Telefone

🟩 RF-A002 Validação de Email

✔ Supabase já faz unicidade

✔ Validação de formato implementada

🟩 RF-A003 Login

✔ Funcionando

🟥 RF-A004 Recuperação de Senha

❗ NÃO implementado ainda

Mas é simples — Supabase tem resetPasswordForEmail

🟨 RF-A005 Editar Perfil do Mercado

✔ Criamos a tela

❗ Falta ajustar navegação e recarregar dados corretamente

❗ Falta permitir alterar logo

🟩 RF-A006 Adicionar Promoção

✔ Tela feita

✔ Upload funcionando

✔ Banco funcionando

🟩 RF-A007 Editar Promoção

✔ Feito

🟩 RF-A008 Remover Promoção

✔ Feito

🟩 RF-A009 Listagem de Promoções

✔ Feito

RF-C — Módulo Cliente Comum (Usuário)

🟩 RF-C001 Acesso sem Login

✔ Já é assim

🟧 RF-C002 Listar Mercados

✔ Já fizemos

❗ Falta melhorar exibição quando não há mercados

🟧 RF-C003 Pesquisar Mercados

✔ Funciona

❗ Falta refinar UI

🟧 RF-C004 Ver Perfil do Mercado

✔ Funciona

❗ Falta exibir endereço/telefone com layout melhor

❗ Falta exibir logo redondo ou estilizado

🟧 RF-C005 Exibir Promoções Ativas

✔ Já funciona

❗ Falta layout final

❗ Falta ordenação por preço ou data

🟩 RF-C006 Detalhes da Promoção

✔ Já temos estrutura

❗ Falta criar a tela final com design

RNF — Requisitos Não Funcionais

RNF001 Flutter Android → ✔

RNF002 Otimização → 🟨 (melhorará com .select(...) otimizados)

RNF003 UI intuitiva → 🟨

RNF004 Segurança de senhas → ✔

RNF005 HTTPS → ✔

RNF006 Storage externo → ✔ Supabase Storage

RNF007 Promoção expirada oculta → ✔

Já filtramos por NOW() entre start e end

📌 Conclusão Geral

👉 O projeto está 85% pronto.

👉 Favoritos realmente não pertence ao escopo e deve ser removido completamente.

👉 As próximas etapas devem seguir a estrutura abaixo:

📍 PRÓXIMAS ETAPAS (na ordem correta)

1️⃣ Remover completamente o módulo de favoritos (não faz parte do projeto)

- Remover buttons ❤️
- Remover FavoritesService
- Remover FavoritesScreen
- Limpar UserHomeScreen (remover aba Favoritos)

2️⃣ Finalizar Módulo Cliente Comum

Tela de Perfil do Mercado (versão final)

- Logo grande
- Endereço
- Telefone
- Lista de promoções ativas

Tela de Detalhes da Promoção (RF-C006)

- Foto grande
- Preço grande
- Descrição
- Validade da promoção
- Nome do mercado
- Botão "Ver mercado"

3️⃣ Revisar Design (UI/UX)

- Ajustar cards
- Ajustar espaçamentos
- Adicionar skeleton loading
- Adicionar Empty states

4️⃣ Implementar Recuperação de Senha (RF-A004)

Com supabase.auth.resetPasswordForEmail().

5️⃣ Testes finais e limpeza

CONCLUSÃO

O ChatGPT conseguiu entregar um MVP simples de um app de catálogo de preços, não foi trabalho questões de UX/UI, e sim as funcionalidade estipuladas na

documentação. Alguns requisitos também não foram desenvolvidos seguindo o fluxo e foi necessário trabalhar neles separadamente e são eles:

RF-A004: (Implícito) O admin deve poder realizar a recuperação de senha (ex: "Esqueci minha senha").
recuperação de senha não foi criado como descrito no documento, foi desenvolvido separadamente com o auxílio da IA.

RF-A006: O admin (logado) deve poder cadastrar um novo produto em promoção, informando: foto do produto, preço promocional, tempo de promoção (data/hora de início e fim) e descrição.
Este recurso foi desenvolvido parcialmente, o tempo de promoção foi feito somente com data.

RF-C006: O usuário deve poder visualizar os detalhes de um produto em promoção (foto, preço, descrição, validade da promoção).
Este recurso foi desenvolvido parcialmente com a ausência da validação da promoção, porém, foram desenvolvidos dois botões, um que leva para o Google Maps usando o endereço cadastrado e outro botão indo para a página do mercado.

UC-04: Gerenciar a aplicação
Superusuário não foi criado como descrito no documento, foi desenvolvido separadamente com o auxílio da IA.

Implementação Recuperação de Senha (RF-A004)
Tive dificuldades para desenvolver essa etapa no Supabase, mas foi possível construir esse recurso, porém só funciona com o aplicativo instalado em um dispositivo fisico, não funciona em emuladores.

Implementar UC-04: Gerenciar Aplicação (Super usuário)

Houve dificuldades na implementação do super usuário nos passos fornecidos foram utilizados dados antigos de funcionamento da adição de meta dados de usuário para que fosse possível o Supabase entender que uma determinada conta de superusuário era de níveis elevados e assim identificar pelo aplicativo e liberar o fluxo de telas especificos pra esse usuário para que ele possa gerenciar os mercados e os produtos.
O que ele diz aqui:
Entendi — isso acontece porque o Supabase mudou a interface em 2024/2025 e agora o painel não permite mais editar app_metadata ou user_metadata diretamente pela tela do usuário.
Mas é possível editar sim — apenas mudou o caminho e a forma.
Existem 3 métodos válidos para adicionar "role": "super".

Deixa claro que estava fornecendo caminhos antigos de resolver o problema. E mesmo com os passos novos ainda não cheguei no resultado esperado, não conseguindo o metaData para adicionar um “marcador” no usuário indicando que ele é “super”. Para resolver isso, no código do Flutter fiz uma verificação somente pelo e-mail para dar permissão de acesso a conta super.

Prompt (Versão 3.0) – 20 de novembro de 2025

Para este prompt tivemos que tentar criar uma solução com o auxílio do Gemini para os erros que o ChatGPT teve, na tentativa de evitar com que eles ocorram novamente e chegar no nosso objetivo final sem a intervenção do usuário ao longo da construção de um projeto Flutter.

Função: Você é um especialista em desenvolvimento mobile Flutter e Supabase (Free Tier).

Restrições:

1. Backend Exclusivo: Supabase (Free Tier) — Apenas Storage e Realtime.
2. Tecnologias Proibidas: Firebase, AWS, ou qualquer tecnologia com planos pagos.
3. Escopo Estrito: Implementar APENAS os requisitos listados na documentação.
4. Conclusão Completa: TODOS os requisitos da documentação (ex: superusuário) devem ser implementados antes da conclusão.

Fluxo de Trabalho Inicial:

1. Confirmação: Confirmar a leitura da documentação (a ser enviada).
2. Validação de Escopo: Listar os 3 requisitos mais críticos (incluindo o superusuário).
3. Aprovação: Solicitar confirmação (Sim/Não) da lista de requisitos antes de prosseguir.

Fluxo de Trabalho Incremental:

1. Estrutura da Etapa: Cada etapa deve ser autocontida e incluir:

a) Título da Etapa.

b) Código completo da Etapa.

c) Orientações detalhadas para o VSCode.

2. Avanço: Perguntar: "Etapa [Número da Etapa] concluída? Podemos avançar?".
3. Continuidade: Avançar somente após a resposta "Sim".
4. Finalização: Ao concluir todos os requisitos, informar que o projeto está pronto, listando os 3 requisitos mais críticos para validação final.

OBSERVAÇÕES

O ChatGPT não possui um padrão de resposta em que um prompt vai produzir sempre a mesma resposta esperada e levar para o mesmo caminho de construção de projeto, fiz várias execuções com o mesmo prompt (versão 3.0) e cada resposta foi diferente uma da outra, algumas com passo mais detalhados e outros menos detalhados, este teste foi feito usando a aba anônima do navegador (Brave) e outros em aba normal usando uma conta com a memória do ChatGPT desligada.

No ultimo prompt testado na primeira etapa houve problemas no auth_service.dart onde a variavel res que faz ligação ao BucketStorage do Supabase não consegue acessar as propriedades como error e data.

Precisamos refatorar o prompt, o ChatGPT está utilizando códigos antigos e pede para sempre atualizar e usar comandos como: flutter pub add nome_pacote que vai sempre baixar versões atualizadas dos pacotes. Por exemplo na Etapa 1 na configuração do supabase_service.dart ele entrega um código de inserção de informações na tabela market onde é chamado um método execute( ) que nas versões atuais do pubase_flutter isso não é mais necessário.

Prompt (Versão 4.0) – 21 de novembro de 2025

Para este prompt tivemos que adicionar uma restrição nova relacionada ao uso de códigos antigos e legados de pacotes do pubspec.yaml.

Função: Você é um especialista em desenvolvimento mobile Flutter e Supabase (Free Tier).

Restrições:

1. Backend Exclusivo: Supabase (Free Tier) — Apenas Storage e Realtime.
2. Tecnologias Proibidas: Firebase, AWS, ou qualquer tecnologia com planos pagos.
3. Escopo Estrito: Implementar APENAS os requisitos listados na documentação.
4. Conclusão Completa: TODOS os requisitos da documentação (ex: superusuário) devem ser implementados antes da conclusão.
5. Código atualizado: SEMPRE desenvolva códigos atuais dos pacotes nas suas últimas versões.

Fluxo de Trabalho Inicial:

1. Confirmação: Confirmar a leitura da documentação (a ser enviada).
2. Validação de Escopo: Listar os 3 requisitos mais críticos (incluindo o superusuário).
3. Aprovação: Solicitar confirmação (Sim/Não) da lista de requisitos antes de prosseguir.

Fluxo de Trabalho Incremental:

1. Estrutura da Etapa: Cada etapa deve ser autocontida e incluir:

a) Título da Etapa.

b) Código completo da Etapa.

c) Orientações detalhadas para o VSCode.

2. Avanço: Perguntar: "Etapa [Número da Etapa] concluída? Podemos avançar?".
3. Continuidade: Avançar somente após a resposta "Sim".
4. Finalização: Ao concluir todos os requisitos, informar que o projeto está pronto, listando os 3 requisitos mais críticos para validação final.

OBSERVAÇÕES

O ChatGPT gerou códigos de versões antigas mas indicou versões mais novas dos pacotes que precisava utilizar e isso causou problemas pela evolução do código entre versões tão distintas.

Prompt (Versão 5.0) – 22 de novembro de 2025

Para este prompt foi necessário evoluir a restrição 5 por que o ChatGPT não estava seguindo o que estava sendo pedido, foram adicionados mais detalhes nesta restrição.

Função: Você é um especialista em desenvolvimento mobile Flutter e Supabase (Free Tier).

Restrições:

1. Backend Exclusivo: Supabase (Free Tier) — Apenas Storage e Realtime.
2. Tecnologias Proibidas: Firebase, AWS, ou qualquer tecnologia com planos pagos.
3. Escopo Estrito: Implementar APENAS os requisitos listados na documentação.
4. Conclusão Completa: TODOS os requisitos da documentação (ex: superusuário) devem ser implementados antes da conclusão.
5. Código atualizado: SEMPRE desenvolva códigos atuais dos pacotes nas suas últimas versões, faça pesquisas na internet e utilize as documentações dos pacotes que vai utilizar. Se atente a versão que está disponibilizando no pubspec na construção do código.

Fluxo de Trabalho Inicial:

1. Confirmação: Confirmar a leitura da documentação (a ser enviada).
2. Validação de Escopo: Listar os 3 requisitos mais críticos (incluindo o superusuário).
3. Aprovação: Solicitar confirmação (Sim/Não) da lista de requisitos antes de prosseguir.

Fluxo de Trabalho Incremental:

1. Estrutura da Etapa: Cada etapa deve ser autocontida e incluir:

a) Título da Etapa.

b) Código completo da Etapa.

c) Orientações detalhadas para o VSCode.

2. Avanço: Perguntar: "Etapa [Número da Etapa] concluída? Podemos avançar?".
3. Continuidade: Avançar somente após a resposta "Sim".
4. Finalização: Ao concluir todos os requisitos, informar que o projeto está pronto, listando os 3 requisitos mais críticos para validação final.

OBSERVAÇÕES

Ainda está acontecendo o mesmo problema anterior. Usando versões de pacotes recentes com códigos antigos.

Prompt (Versão 6.0) – 22 de novembro de 2025

Para este prompt foi necessário evoluir a restrição 5 por que o ChatGPT não estava seguindo o que estava sendo pedido, foram adicionados mais detalhes nesta restrição, com exemplos para que ele possa se basear e não cometer os mesmos erros.

Função: Você é um especialista em desenvolvimento mobile Flutter e Supabase (Free Tier).

Restrições:

1. Backend Exclusivo: Supabase (Free Tier) — Apenas Storage e Realtime.
2. Tecnologias Proibidas: Firebase, AWS, ou qualquer tecnologia com planos pagos.
3. Escopo Estrito: Implementar APENAS os requisitos listados na documentação.
4. Conclusão Completa: TODOS os requisitos da documentação (ex: superusuário) devem ser implementados antes da conclusão.
5. Código atualizado: SEMPRE desenvolva códigos atuais dos pacotes nas suas últimas versões, faça pesquisas na internet e utilize as documentações dos pacotes que vai utilizar. Exemplo de erro que não deve ser cometido:

Nas versões superiores a 2.10.2 (essa versão é só um exemplo) do pacote supabase_flutter não existe mais o método execute( ) na inserção de dados nas tabelas do Supabase.

Fluxo de Trabalho Inicial:

1. Confirmação: Confirmar a leitura da documentação (a ser enviada).
2. Validação de Escopo: Listar os 3 requisitos mais críticos (incluindo o superusuário).
3. Aprovação: Solicitar confirmação (Sim/Não) da lista de requisitos antes de prosseguir.

Fluxo de Trabalho Incremental:

1. Estrutura da Etapa: Cada etapa deve ser autocontida e incluir:

a) Título da Etapa.

b) Código completo e detalhado da Etapa.

c) Orientações detalhadas para o VSCode.

2. Avanço: Perguntar: "Etapa [Número da Etapa] concluída? Podemos avançar?".
3. Continuidade: Avançar somente após a resposta "Sim".
4. Finalização: Ao concluir todos os requisitos, informar que o projeto está pronto, listando os 3 requisitos mais críticos para validação final.

OBSERVAÇÕES

A versão 3.0 foi gerada pelo Gemini com base nos relatos de dificuldades com os prompts até então. Nesse mesmo momento, foi gerada uma mesma versão no ChatGPT relatando as mesmas dificuldades (ou seja, o mesmo prompt que pediu um novo prompt para o Gemini foi usado no ChatGPT). Da versão 3.0 até a 6.0 foram evoluções da versão do Gemini que chegou a um limite. A partir da versão 7.0 é a versão gerada pelo ChatGPT incluindo as dificuldades relatadas entre a versão 3 e a versão 6 do prompt.

Prompt (Versão 7.0) – 25 de novembro de 2025

Use exclusivamente o documento:
/mnt/data/documento_economizapp.odt

Implemente o EconomizApp exatamente como descrito, usando Flutter + Supabase (Auth, Postgres, Storage).
Não usar Firebase. Não adicionar funcionalidades não previstas.

REGRAS PRINCIPAIS

1. Leia o documento todo e gere primeiro um resumo dos requisitos (RF, RNF, casos de uso, atores).
2. Não implemente nada que não esteja no documento. Se quiser sugerir algo extra, coloque apenas em “Opcional (não implementar)”.
3. Qualquer item mal definido → liste em Ambiguidades e faça uma suposição clara.
4. O superusuário deve ser implementado obrigatoriamente (inclusive seed).
5. Imagens sempre no Supabase Storage; banco guarda apenas URLs.
6. Nenhuma rota, botão, ação, tela, função ou serviço pode ser gerado sem ligação funcional.

REGRAS CONTRA CÓDIGO DESATUALIZADO

- Use sempre a versão estável mais recente do Flutter e Dart, com null safety.
- Gere código apenas com APIs atuais.

- Proibido usar itens deprecated (ex.: `RaisedButton`, `FlatButton`, APIs antigas do Supabase).
- Antes de cada trecho de código, escreva:
  "APIs verificadas: sem deprecated"
- Se detectar algo desatualizado, corrija automaticamente.

REGRAS CONTRA BOTÕES SEM AÇÃO

- Nenhum widget interativo pode ser entregue sem callback funcional (`onPressed`, `onTap`, etc.).
- Antes de cada tela, liste todos os botões e suas ações e confirme:
  "Tela validada: todos os botões possuem callbacks funcionais"
- Nenhum serviço, função ou rota pode ser referenciado sem estar implementado.
- Se a lógica ainda não puder ser implementada, criar stub funcional (nunca deixar vazio).

MODO INCREMENTAL

Você só avança quando eu disser:

- PRÓXIMA → ir para a próxima fase
- AUTONOMO → executar todas as fases até o final sem pausas

FASES

1. Requisitos + ambiguidades + modelo de dados
2. Ambiente (Flutter + VSCode + Supabase)
3. Backend (SQL, policies, superusuário, storage)
4. Estrutura do app Flutter
5. Funcionalidades previstas no documento
6. Autenticação, permissões e superusuário
7. Checklist final (cada RF → ✓ ou ✗)
8. PROJETO CONCLUÍDO

FORMATO DE ENTREGA

- Código completo, atual e funcional, organizado por arquivos
- Comandos de terminal quando necessários
- Sem adicionar nada além do que está no documento

INTERVENÇÕES

1. O erro `FileNotFoundError` (ou `Unable to load asset`) ao usar `flutter_dotenv` ocorre quase sempre por um motivo específico: o Flutter não reconhece o arquivo `.env` como um asset empacotado na aplicação.

O ChatGPT não informou que era necessário utilizar:

assets: - .env no arquivo pubspec.yaml para resolver este problema.

2. Fez o uso do pacote **image_picker** na criação da tela **signup_market_page.dart** e **market_details_page.dart** mas não informou que era necessário utilizar o comando **flutter pub add image_picker** para utilizar os recursos do pacote.

3. No arquivo market_profile_page.dart na variável String? *marketId,* onde o ? indica que essa variável é do tipo nullable*,* esse detalhe faz parte do *null*safety para que o Flutter tenha conhecimento que uma determinada variável pode ser nula e devemos tratar ela para que o app não quebre. O erro no ChatGPT referente a variável marketId é que como ele usa uma variável possivelmente nula em alguns trechos do código em que ela é utilizada dentro do parâmetro de função deve se colocar ! ao lado direito da variável (marketId!) para indicar ao Flutter que ela não vai gerar null em nenhum momento, mas o ChatGPT não colocou essa indicação o que gera erro. Mesmo problema também aconteceu com o products_details_page.dart na variável _productId que do tipo *nullable* (?).

CONCLUSÃO

Com esse prompt foi possível finalizar o projeto, mas não foi possível gerar um MVP funcional. Erros no Supabase referentes a cadastro de produtos, listagem de produtos, carregamentos dos dados do mercado. O Superusuário foi tratado como um mercado comum e não um ator que é responsável por gerenciar toda a aplicação.

OBSERVAÇÕES

No desenvolvimento do projeto com o prompt anterior o ChatGPT seguiu passos usando o Supabase Desktop, esse software utiliza o Docker para salvar o BD localmente nos containers. Estou usando Linux e tive problema de permissões ao utilizar o Docker e não consegui avançar, por isso agora pedi para utilizar exclusivamente o Supabase Web para que conseguissemos avançar melhor.

Prompt (Versão 8.0) – 26 de novembro de 2025
Use exclusivamente o documento:
/mnt/data/documento_economizapp.odt

Implemente o EconomizApp exatamente como descrito, usando Flutter + Supabase (Auth, Postgres, Storage).
Não usar Firebase. Não adicionar funcionalidades não previstas.
Utilizar somente o Supabase Web para acessar as funcionalidades dele, sempre entregar passos atualizados do site.

REGRAS PRINCIPAIS

7. Leia o documento todo e gere primeiro um resumo dos requisitos (RF, RNF, casos de uso, atores).
8. Não implemente nada que não esteja no documento. Se quiser sugerir algo extra, coloque apenas em “Opcional (não implementar)”.
9. Qualquer item mal definido → liste em Ambiguidades e faça uma suposição clara.
10. O superusuário deve ser implementado obrigatoriamente (inclusive seed).
11. Imagens sempre no Supabase Storage; banco guarda apenas URLs.
12. Nenhuma rota, botão, ação, tela, função ou serviço pode ser gerado sem ligação funcional.

REGRAS CONTRA CÓDIGO DESATUALIZADO

- Use sempre a versão estável mais recente do Flutter e Dart, com null safety.
- Gere código apenas com APIs atuais.

- Proibido usar itens deprecated (ex.: `RaisedButton`, `FlatButton`, APIs antigas do Supabase).
- Antes de cada trecho de código, escreva:
  "APIs verificadas: sem deprecated"
- Se detectar algo desatualizado, corrija automaticamente.

REGRAS CONTRA BOTÕES SEM AÇÃO

- Nenhum widget interativo pode ser entregue sem callback funcional (`onPressed`, `onTap`, etc.).
- Antes de cada tela, liste todos os botões e suas ações e confirme:
  "Tela validada: todos os botões possuem callbacks funcionais"
- Nenhum serviço, função ou rota pode ser referenciado sem estar implementado.
- Se a lógica ainda não puder ser implementada, criar stub funcional (nunca deixar vazio).

MODO INCREMENTAL

Você só avança quando eu disser:

- PRÓXIMA → ir para a próxima fase
- AUTONOMO → executar todas as fases até o final sem pausas

FASES

9. Requisitos + ambiguidades + modelo de dados
10. Ambiente (Flutter + VSCode + Supabase)
11. Backend (SQL, policies, superusuário, storage)
12. Estrutura do app Flutter
13. Funcionalidades previstas no documento
14. Autenticação, permissões e superusuário
15. Checklist final (cada RF → ✓ ou ✗)
16. PROJETO CONCLUÍDO

FORMATO DE ENTREGA

- Código completo, atual e funcional, organizado por arquivos
- Comandos de terminal quando necessários
- Sem adicionar nada além do que está no documento

CONCLUSÃO (prompt v8.0)

ChatGPT finalizou o projeto com vários erros vindos do Supabase no cadastro dos produtos, também não foi criado o recurso de escolher uma imagem do mercado e do produto, o recurso de editar os dados de um mercado também não foi feito. Foi preciso intervir no final por que não foi criado alguma interação que o usuário possa fazer e foi realizado uma interação, mas vários problemas relatados aconteceram. Outra questão que aconteceu no fim da construção do projeto que colocarei a resposta.

- superusuário pode ler mercados (policy anon permite)
- superusuário pode ler produtos (anon NÃO pode).
- superusuário pode deletar qualquer produto (não permitido pelas policies).
- superusuário pode deletar mercados (não permitido).

Portanto o superusuário **precisa de sessão com service_role** para bypass RLS.

Mas o documento exige:
"Utilizar apenas Supabase Web e Auth padrão (email/senha)"
✔ Não permite adicionar backend extra.
✔ Não permite usar service_role no cliente.

Portanto:

📌 **Implementação mínima necessária (aceita pelo documento):**
Superusuário só pode **listar** mercados e produtos (não deletar), pois deletion universal não está previsto no documento original (UC-04 não detalha poder deletar).

E o documento só diz:

“Superusuário com controle total sobre a aplicação”
mas **não especifica ações**, portanto deletar mercados/produtos é uma **suponção**, não obrigação.

Conclusão: **Remover operação de delete do superusuário**, pois ela não é prevista e viola RLS.

A última modificação no prompt: “Utilizar somente o Supabase Web para acessar as funcionalidades dele, sempre entregar passos atualizados do site” fez com que o ChatGPT não desenvolvesse os recursos como ter autonomia para deletar mercados e produtos, por que existem as políticas que ele mesmo criou para questões de segurança no backend, então é dito que teria que criar um bypass nas políticas por que o super usuário foi criado em sua tabela própria e ter capacidades de modificar (excluir, editar) outras tabelas infringem as políticas. A solução talvez seja remover a obrigatoriedade de uso do Supabase Web.

OBSERVAÇÕES

Na última experiência com o prompt versão 8.0 tivemos problemas com o Supabase Web e também com o super usuário e o que ele pode gerenciar, por isso, houve mudanças no documento do projeto para que o ChatGPT entenda de forma clara o que o super usuário faz. Este prompt é o mesmo da versão 7.0, porém agora a documentação deixa explícito as funcionalidades do super usuário. Também será retirado o Supabase Web e trabalhar localmente usando Supabase CLI.

Prompt (Versão 9.0) – 26 de novembro de 2025

Use exclusivamente o documento:
/mnt/data/documento_economizapp.odt

Implemente o EconomizApp exatamente como descrito, usando Flutter + Supabase (Auth, Postgres, Storage).
Não usar Firebase. Não adicionar funcionalidades não previstas.
Utilizar o Supabase CLI para trabalharmos localmente.

REGRAS PRINCIPAIS

13. Leia o documento todo e gere primeiro um resumo dos requisitos (RF, RNF, casos de uso, atores).
14. Não implemente nada que não esteja no documento. Se quiser sugerir algo extra, coloque apenas em “Opcional (não implementar)”.
15. Qualquer item mal definido → liste em Ambiguidades e faça uma suposição clara.
16. O superusuário deve ser implementado obrigatoriamente (inclusive seed).
17. Imagens sempre no Supabase Storage; banco guarda apenas URLs.
18. Nenhuma rota, botão, ação, tela, função ou serviço pode ser gerado sem ligação funcional.

REGRAS CONTRA CÓDIGO DESATUALIZADO

- Use sempre a versão estável mais recente do Flutter e Dart, com null safety.
- Gere código apenas com APIs atuais.

- Proibido usar itens deprecated (ex.: `RaisedButton`, `FlatButton`, APIs antigas do Supabase).
- Antes de cada trecho de código, escreva:
  "APIs verificadas: sem deprecated"
- Se detectar algo desatualizado, corrija automaticamente.

REGRAS CONTRA BOTÕES SEM AÇÃO

- Nenhum widget interativo pode ser entregue sem callback funcional (`onPressed`, `onTap`, etc.).
- Antes de cada tela, liste todos os botões e suas ações e confirme:
  "Tela validada: todos os botões possuem callbacks funcionais"
- Nenhum serviço, função ou rota pode ser referenciado sem estar implementado.
- Se a lógica ainda não puder ser implementada, criar stub funcional (nunca deixar vazio).

MODO INCREMENTAL

Você só avança quando eu disser:

- PRÓXIMA → ir para a próxima fase
- AUTONOMO → executar todas as fases até o final sem pausas

FASES

17. Requisitos + ambiguidades + modelo de dados
18. Ambiente (Flutter + VSCode + Supabase)
19. Backend (SQL, policies, superusuário, storage)
20. Estrutura do app Flutter
21. Funcionalidades previstas no documento
22. Autenticação, permissões e superusuário
23. Checklist final (cada RF → ✓ ou ✗)
24. PROJETO CONCLUÍDO

FORMATO DE ENTREGA

- Código completo, atual e funcional, organizado por arquivos
- Comandos de terminal quando necessários
- Sem adicionar nada além do que está no documento

CONCLUSÃO (prompt v9.0)

Na construção do aplicativo usando Supabase Local houve problemas em relação ao uso do emulador para execução do projeto onde o ip local no emulador é diferente e não teve sucesso, tentativas de compilar o código para web e app linux também foram fracassadas. Requisitos de super usuário como: excluir mercado e produtos do mercados foram ignorados no documento mesmo estando presentes.

RESULTADO FINAL (CHATGPT)

O ChatGPT se mostrou ineficaz em seguir estritamente a documentação, ignorando recursos que eram para ser criados, como por exemplo: no último prompt v9.0 foi utilizado na documentação do projeto que o super usuário teria a capacidade de gerenciar totalmente os mercados e seus produtos que incluem a edição e exclusão deles e durante o processo de desenvolvimento foi dito que a exclusão não estava listada como um recurso importante. Em vários momentos da construção de telas e recursos para o superusuário era enfatizado que o recurso de exclusão estava presente, por isso, não seria desenvolvido. A conclusão com o ChatGPT é de que essa IA não lida bem com documentações e isso foi percebido ao longo de todo o percurso do desenvolvimento. Outra questão está na falta de clareza em alguns passos por que foi acompanhado cada passo que a IA construía e nos últimos prompts ela dava a opção de ser autônoma, ou seja, já desenvolver todos os passos de uma vez só.

Informações importantes

Os testes a partir de agora estão sendo realizados usando o Gemini como desenvolvedor de código, em uma conta gratuita e usando a versão 3 PRO. Vamos recomeçar usando o prompt versão 7.0. Lembrando que a documentação do projeto foi atualizada para deixar mais claro as funções do super usuário.

Gemini

Prompt (Versão 7.0) – 29 de novembro de 2025

Use exclusivamente o documento:
/mnt/data/documento_economizapp_2.0.odt

Implemente o EconomizApp exatamente como descrito, usando Flutter + Supabase (Auth, Postgres, Storage).
Não usar Firebase. Não adicionar funcionalidades não previstas.

REGRAS PRINCIPAIS

19. Leia o documento todo e gere primeiro um resumo dos requisitos (RF, RNF, casos de uso, atores).
20. Não implemente nada que não esteja no documento. Se quiser sugerir algo extra, coloque apenas em “Opcional (não implementar)”.
21. Qualquer item mal definido → liste em Ambiguidades e faça uma suposição clara.
22. O superusuário deve ser implementado obrigatoriamente (inclusive seed).
23. Imagens sempre no Supabase Storage; banco guarda apenas URLs.
24. Nenhuma rota, botão, ação, tela, função ou serviço pode ser gerado sem ligação funcional.

REGRAS CONTRA CÓDIGO DESATUALIZADO

- Use sempre a versão estável mais recente do Flutter e Dart, com null safety.
- Gere código apenas com APIs atuais.

- Proibido usar itens deprecated (ex.: `RaisedButton`, `FlatButton`, APIs antigas do Supabase).
- Antes de cada trecho de código, escreva:
  "APIs verificadas: sem deprecated"
- Se detectar algo desatualizado, corrija automaticamente.

REGRAS CONTRA BOTÕES SEM AÇÃO

- Nenhum widget interativo pode ser entregue sem callback funcional (`onPressed`, `onTap`, etc.).
- Antes de cada tela, liste todos os botões e suas ações e confirme:
  "Tela validada: todos os botões possuem callbacks funcionais"
- Nenhum serviço, função ou rota pode ser referenciado sem estar implementado.
- Se a lógica ainda não puder ser implementada, criar stub funcional (nunca deixar vazio).

MODO INCREMENTAL

Você só avança quando eu disser:

- PRÓXIMA → ir para a próxima fase
- AUTONOMO → executar todas as fases até o final sem pausas

FASES

25. Requisitos + ambiguidades + modelo de dados
26. Ambiente (Flutter + VSCode + Supabase)
27. Backend (SQL, policies, superusuário, storage)
28. Estrutura do app Flutter
29. Funcionalidades previstas no documento
30. Autenticação, permissões e superusuário
31. Checklist final (cada RF → ✓ ou ✗)
32. PROJETO CONCLUÍDO

FORMATO DE ENTREGA

- Código completo, atual e funcional, organizado por arquivos
- Comandos de terminal quando necessários
- Sem adicionar nada além do que está no documento

Desenvolvimento

O Gemini se organizou na criação do projeto em etapas, porém na etapa 4 ele atingiu o limite de uso para o Gemini 3 PRO, então, temos duas opções:

1 - Esperar várias horas e poder voltar a utilizar a versão PRO.

2 - Continuar usando com um modelo mais antigo.

Como estamos utilizando uma conta gratuita, e não podemos esperar toda vez que o limite de uso da versão PRO é atingido, a decisão correta é continuar o desenvolvimento em um modelo inferior chamado de rápido mas com pouco nível de complexidade.

Conclusão

Na finalização das etapas e conclusão do projeto apresentaram-se vários problemas ligados ao serviço do Supabase e suas políticas para segurança do banco de dados.

1. Erro ao cadastrar um novo mercado: Depois de preencher os dados do mercado e clicar no botão botão de cadastrar um erro acontece da seguinte forma: A nova linha viola a segurança de políticas de nível baixo, dizendo que não é autorizado com erro 403. Ou seja, é um erro que fere as próprias políticas que o Gemini criou. Questionando o Gemini ele diz o seguinte:

   Este erro `StorageException(message: new row violates row-level security policy, statusCode: 403, error: Unauthorized)` indica que o problema não está no código Flutter, mas sim em uma **Política de Segurança em Nível de Linha (RLS)** do **Supabase Storage**.

   O fluxo de cadastro do mercado tenta primeiro criar o usuário (Auth) e, em seguida, fazer o upload da logo para o bucket `market-logos`. Embora o

usuário esteja autenticado, a política RLS definida para o upload de arquivos é muito genérica e não valida a propriedade do arquivo, o que causa o erro 403.

A solução fornecida envolveu a tentativa de modificar as políticas para corrigir o problema de uploud das logos, mas, não foi possível resolver esse problema por que o erro continuou a existir mesmo após as sugestões de solução dadas.

Foi pedido que o Gemini removesse todas as políticas de segurança para fins de teste e mesmo assim não foi possível cadastrar o mercado com os mesmos erros mencionados anteriormente.

2. É possível logar no usuário (mercado) criado, porém, esse usuário consegue se autenticar, mas, seus dados foram incluídos na tabela de mercado e com isso, não consigo editar e também não consigo cadastrar um produto.

Finalizando, é perceptível que o Gemini parece não lidar tão bem com a construção do lado do servidor, mas interfaces, toda a parte gráfica foi possívelmente construída corretamente, porque não podemos avançar em algumas telas por conter os relatados.

OBSERVAÇÕES

As mudanças de prompt estão relacionadas às políticas de segurança do Supabase que provavelmente são as responsáveis por causar problemas no cadastro do mercado e consequentemente nos cadastros dos produtos. Por isso, foi adicionado comandos para a não utilização das políticas de segurança do Supabase.

Prompt (Versão 10.0) – 29 de novembro de 2025

Use exclusivamente o documento:
/mnt/data/documento_economizapp_2.0.odt

Implemente o EconomizApp exatamente como descrito, usando Flutter + Supabase (Auth, Postgres, Storage).
Não usar Firebase. Não adicionar funcionalidades não previstas.
Não trabalhar com políticas de segurança, nos buckets, nas tabelas entre outros, porque primeiro devemos testar se tudo está funcionando e depois trabalharemos na parte de segurança.

REGRAS PRINCIPAIS

25. Leia o documento todo e gere primeiro um resumo dos requisitos (RF, RNF, casos de uso, atores).
26. Não implemente nada que não esteja no documento. Se quiser sugerir algo extra, coloque apenas em “Opcional (não implementar)”.
27. Qualquer item mal definido → liste em Ambiguidades e faça uma suposição clara.
28. O superusuário deve ser implementado obrigatoriamente (inclusive seed).
29. Imagens sempre no Supabase Storage; banco guarda apenas URLs.
30. Nenhuma rota, botão, ação, tela, função ou serviço pode ser gerado sem ligação funcional.

REGRAS CONTRA CÓDIGO DESATUALIZADO

- Use sempre a versão estável mais recente do Flutter e Dart, com null safety.
- Gere código apenas com APIs atuais.

- Proibido usar itens deprecated (ex.: `RaisedButton`, `FlatButton`, APIs antigas do Supabase).
- Antes de cada trecho de código, escreva:
  "APIs verificadas: sem deprecated"
- Se detectar algo desatualizado, corrija automaticamente.

REGRAS CONTRA BOTÕES SEM AÇÃO

- Nenhum widget interativo pode ser entregue sem callback funcional (`onPressed`, `onTap`, etc.).
- Antes de cada tela, liste todos os botões e suas ações e confirme:
  "Tela validada: todos os botões possuem callbacks funcionais"
- Nenhum serviço, função ou rota pode ser referenciado sem estar implementado.
- Se a lógica ainda não puder ser implementada, criar stub funcional (nunca deixar vazio).

MODO INCREMENTAL

Você só avança quando eu disser:

- PRÓXIMA → ir para a próxima fase
- AUTONOMO → executar todas as fases até o final sem pausas

FASES

33. Requisitos + ambiguidades + modelo de dados
34. Ambiente (Flutter + VSCode + Supabase)
35. Backend (SQL, policies, superusuário, storage)
36. Estrutura do app Flutter
37. Funcionalidades previstas no documento
38. Autenticação, permissões e superusuário
39. Checklist final (cada RF → ✓ ou ✗)
40. PROJETO CONCLUÍDO

FORMATO DE ENTREGA

- Código completo, atual e funcional, organizado por arquivos
- Comandos de terminal quando necessários
- Sem adicionar nada além do que está no documento

CONCLUSÃO

Na finalização das etapas e a conclusão do projeto foram detectados diversos problemas, principalmente com o uso dos buckets que são responsáveis por armazenar as imagens, por algum motivo o Supabase não estava permitindo o armazenamento das imagens. Foram criados dois buckets, um chamado logos para armazenar as logos dos mercados e outro chamado products para armazenar as imagens dos produtos, e nenhum dos dois funcionou. Testes foram feitos sem o armazenamento das imagens e foi constatado que era esse o problema. A solução para as imagens dos buckets veio na criação de políticas de segurança para os usuários autenticados por que o Supabase é muito restritivo com operações de inserção e não definir quem pode ou não escrever é importante. O Super usuário novamente não teve seus recursos implementados, como edição de mercados e possibilidade exclusão, apenas listagem de mercados. Foram removidas todas as políticas de segurança para fins de teste. O super usuário foi interpretado como um mercado e não como um papel de gerenciador da plataforma, para ele ficou disponível cadastrar produtos.

OBSERVAÇÕES

Atualizações no prompt estão ligadas às políticas de segurança (RLS) por que no prompt anterior foi pedido que não houvesse políticas de segurança, mas, nos buckets é preciso dizer exatamente quem pode ou não armazenar as imagens do projeto.

Prompt (Versão 11.0) – 01 de dezembro de 2025

Use exclusivamente o documento:
/mnt/data/documento_economizapp_2.0.odt

Implemente o EconomizApp exatamente como descrito, usando Flutter + Supabase (Auth, Postgres, Storage).
Não usar Firebase. Não adicionar funcionalidades não previstas.
Trabalhar apenas com políticas básicas (RLS) para o funcionamento correto do projeto.

REGRAS PRINCIPAIS

31. Leia o documento todo e gere primeiro um resumo dos requisitos (RF, RNF, casos de uso, atores).
32. Não implemente nada que não esteja no documento. Se quiser sugerir algo extra, coloque apenas em “Opcional (não implementar)”.
33. Qualquer item mal definido → liste em Ambiguidades e faça uma suposição clara.
34. O superusuário deve ser implementado obrigatoriamente (inclusive seed).
35. Imagens sempre no Supabase Storage; banco guarda apenas URLs.
36. Nenhuma rota, botão, ação, tela, função ou serviço pode ser gerado sem ligação funcional.

REGRAS CONTRA CÓDIGO DESATUALIZADO

- Use sempre a versão estável mais recente do Flutter e Dart, com null safety.
- Gere código apenas com APIs atuais.

- Proibido usar itens deprecated (ex.: `RaisedButton`, `FlatButton`, APIs antigas do Supabase).
- Antes de cada trecho de código, escreva:
  "APIs verificadas: sem deprecated"
- Se detectar algo desatualizado, corrija automaticamente.

REGRAS CONTRA BOTÕES SEM AÇÃO

- Nenhum widget interativo pode ser entregue sem callback funcional (`onPressed`, `onTap`, etc.).
- Antes de cada tela, liste todos os botões e suas ações e confirme:
  "Tela validada: todos os botões possuem callbacks funcionais"
- Nenhum serviço, função ou rota pode ser referenciado sem estar implementado.
- Se a lógica ainda não puder ser implementada, criar stub funcional (nunca deixar vazio).

MODO INCREMENTAL

Você só avança quando eu disser:

- PRÓXIMA → ir para a próxima fase
- AUTONOMO → executar todas as fases até o final sem pausas

FASES

41. Requisitos + ambiguidades + modelo de dados
42. Ambiente (Flutter + VSCode + Supabase)
43. Backend (SQL, policies, superusuário, storage)
44. Estrutura do app Flutter
45. Funcionalidades previstas no documento
46. Autenticação, permissões e superusuário
47. Checklist final (cada RF → ✓ ou ✗)
48. PROJETO CONCLUÍDO

FORMATO DE ENTREGA

- Código completo, atual e funcional, organizado por arquivos
- Comandos de terminal quando necessários
- Sem adicionar nada além do que está no documento

CONCLUSÃO

Com esse prompt, o resultado novamente está relacionado ao Supabase e problemas de políticas de segurança (RLS) e também aconteceram problemas no projeto Flutter, ao tentar cadastrar erros relacionado a segurança do Supabase aconteceram, e se o usuário tentasse confirmar o email com o link enviado é possível logar, porém, como os dados do usuário não foram armazenados na tabela de mercado aconteceram erros ao editar e criar produtos.

OBSERVAÇÕES

Para o próximo prompt será feito na estrutura XML, pois ele proporciona uma clareza e mais confiabilidade e veremos se isso ajuda no desenvolvimento.

Prompt (Versão 12.0) – 01 de dezembro de 2025


```
<Prompt>
  <Meta>
    <Version>11.0</Version>
    <Date>2025-12-01</Date>
  </Meta>

  <Source>
    <File>/mnt/data/documento_economizapp_2.0.odt</File>
    <Instruction>Use exclusivamente o documento indicado.</Instruction>
  </Source>

  <Goal>
    Implemente o EconomizApp exatamente como descrito no documento-fonte,
    utilizando Flutter e Supabase (Auth, Postgres, Storage).
    Não usar Firebase. Não adicionar funcionalidades não previstas.
    Aplicar apenas políticas básicas (RLS) necessárias para o funcionamento correto.
  </Goal>

  <Rules>
    <Main>
      <Rule>Ler o documento por completo antes de iniciar.</Rule>
          <Rule>Gerar primeiro um resumo de requisitos (RF, RNF, casos de uso, atores).</Rule>
      <Rule>Não implementar nada que não esteja presente no documento.</Rule>
                <Rule>Itens extras devem ir apenas em &quot;Opcional (não implementar)&quot;.</Rule>
        <Rule>Itens mal definidos devem ser listados em &quot;Ambiguidades&quot; com suposições explícitas.</Rule>
```

<Rule>O superusuário deve ser implementado obrigatoriamente, incluindo seed.</Rule>

<Rule>Imagens devem ser armazenadas no Supabase Storage; o banco salva apenas URLs.</Rule>

<Rule>Nenhuma rota, botão, ação, tela, função ou serviço pode existir sem funcionalidade real.</Rule>

</Main>

<AntiOutdatedCode>

<Rule>Usar sempre a versão estável mais recente de Flutter e Dart, com null safety.</Rule>

<Rule>Gerar código apenas com APIs atuais.</Rule>

<Rule>Proibido usar APIs deprecated.</Rule>

<Rule>Antes de cada trecho de código, escrever: "APIs verificadas: sem deprecated".</Rule>

<Rule>Se for detectado algo desatualizado, corrigir automaticamente.</Rule>

</AntiOutdatedCode>

<InteractiveWidgets>

<Rule>Nenhum widget interativo pode ser entregue sem callback funcional.</Rule>

<Rule>Antes de cada tela, listar todos os botões e ações.</Rule>

<Rule>Confirmar: "Tela validada: todos os botões possuem callbacks funcionais".</Rule>

<Rule>Nenhuma função, rota ou serviço pode ser referenciado sem implementação.</Rule>

<Rule>Se ainda não implementável, criar stub funcional (nunca vazio).</Rule>

</InteractiveWidgets>

<ExecutionMode>

<Mode keyword="PRÓXIMA">Avança para a próxima fase somente quando solicitado.</Mode>

<Mode keyword="AUTONOMO">Executa todas as fases até o final sem pausas.</Mode>

```
    </ExecutionMode>
  </Rules>

  <Phases>
    <Phase>Requisitos, ambiguidades e modelo de dados</Phase>
    <Phase>Ambiente (Flutter + VSCode + Supabase)</Phase>
    <Phase>Backend (SQL, policies, superusuário, storage)</Phase>
    <Phase>Estrutura do app Flutter</Phase>
    <Phase>Funcionalidades previstas no documento</Phase>
    <Phase>Autenticação, permissões e superusuário</Phase>
    <Phase>Checklist final (cada RF → ✓ ou ✗)</Phase>
    <Phase>PROJETO CONCLUÍDO</Phase>
  </Phases>

  <OutputFormat>
    <Rule>Entregar código completo, atual e funcional.</Rule>
    <Rule>Organizar por arquivos conforme estrutura recomendada.</Rule>
    <Rule>Incluir comandos de terminal quando necessários.</Rule>
    <Rule>Não adicionar nada além do que está no documento-fonte.</Rule>
  </OutputFormat>
</Prompt>
```

CONCLUSÃO

Com esse novo formato de prompt usando XML os problemas com o Supabase continuaram, mas aparentemente não tem problemas com o código Dart/Flutter.

Informações importantes

No último prompt, na versão 12.0 com o Gemini aconteceram problemas com o Supabase relacionado às políticas de segurança RLS onde não era possível cadastrar mercados e nem os produtos, o Super usuário foi tratado como um mercado administrador e isso também ficou errado.

Claude

Observações

A próxima IA escolhida foi o Claude no modelo Sonnet 4.5 que é gratuito porém tem seu uso limitado e quando atinge esse limite só poderá ser utilizado novamente depois de algumas horas. Foi utilizado o mesmo prompt versão 12.0 no formato de XML.

Conclusão

O Claude gerou um MVP bem sucedido do projeto, só existiu uma intervenção para correção de código logo no início do desenvolvimento. O projeto final teve problemas no cadastro do primeiro mercado, novamente associado às políticas de segurança RLS envolvendo os buckets de imagens, porém, na tentativa de cadastro do mercado sem utilizar imagem, somente os dados do mercado, foi possível efetuar o cadastro por que não era obrigatório o uso de imagem de mercado para a conclusão e para facilitar os testes foi removido a confirmação de email de novas contas por link. Outras IAs testadas (ChatGPT e Gemini) obrigavam o usuário a cadastrar uma imagem de mercado e nessas versões existiam também problemas com o RLS do Supabase. Efetuando o cadastro do mercado sem imagem no Claude, e depois na edição no próprio aplicativo, colocar uma imagem, funcionou normalmente salvando-a no bucket. Foi possível cadastrar mercadorias com dia e hora de início e de término da promoção. O usuário comum consegue ver os mercados e suas mercadorias. O super usuário consegue ter seu papel de gerenciar(excluir e modificar) informações dos mercados e seus produtos, nas outras IAs isso não estava sendo possível, onde elas muitas vezes confundiam o SUper usuário como mercado comum, apesar que no Claude o super usuário fica listado como sendo um mercado mas ao logar ele não ter características de um mercado comum, mas sim de um gerenciador dos mercados. Sendo assim, o

resultado mais próximo a documentação e um MVP funcional até agora está sendo com o Claude com seu modelo Sonnet 4.5.

Observações

Nesta etapa do desenvolvimento, será utilizado o prompt v.11 em três IAs (ChatGPT, Gemini e Claude) e será analisado a quantidade de interações feitas em cada uma delas. Foi removida a confirmação de email para fins de teste.

Prompt (Versão 11.0) – 01 de dezembro de 2025

Use exclusivamente o documento:
/mnt/data/documento_economizapp_2.0.odt

Implemente o EconomizApp exatamente como descrito, usando Flutter + Supabase (Auth, Postgres, Storage).

Não usar Firebase. Não adicionar funcionalidades não previstas.

Trabalhar apenas com políticas básicas (RLS) para o funcionamento correto do projeto.

REGRAS PRINCIPAIS

37. Leia o documento todo e gere primeiro um resumo dos requisitos (RF, RNF, casos de uso, atores).
38. Não implemente nada que não esteja no documento. Se quiser sugerir algo extra, coloque apenas em “Opcional (não implementar)”.
39. Qualquer item mal definido → liste em Ambiguidades e faça uma suposição clara.
40. O superusuário deve ser implementado obrigatoriamente (inclusive seed).
41. Imagens sempre no Supabase Storage; banco guarda apenas URLs.
42. Nenhuma rota, botão, ação, tela, função ou serviço pode ser gerado sem ligação funcional.

REGRAS CONTRA CÓDIGO DESATUALIZADO

- Use sempre a versão estável mais recente do Flutter e Dart, com null safety.
- Gere código apenas com APIs atuais.

- Proibido usar itens deprecated (ex.: `RaisedButton`, `FlatButton`, APIs antigas do Supabase).

- Antes de cada trecho de código, escreva:
  "APIs verificadas: sem deprecated"
- Se detectar algo desatualizado, corrija automaticamente.

REGRAS CONTRA BOTÕES SEM AÇÃO

- Nenhum widget interativo pode ser entregue sem callback funcional (`onPressed`, `onTap`, etc.).
- Antes de cada tela, liste todos os botões e suas ações e confirme:
  "Tela validada: todos os botões possuem callbacks funcionais"
- Nenhum serviço, função ou rota pode ser referenciado sem estar implementado.
- Se a lógica ainda não puder ser implementada, criar stub funcional (nunca deixar vazio).

MODO INCREMENTAL

Você só avança quando eu disser:

- PRÓXIMA → ir para a próxima fase
- AUTONOMO → executar todas as fases até o final sem pausas

FASES

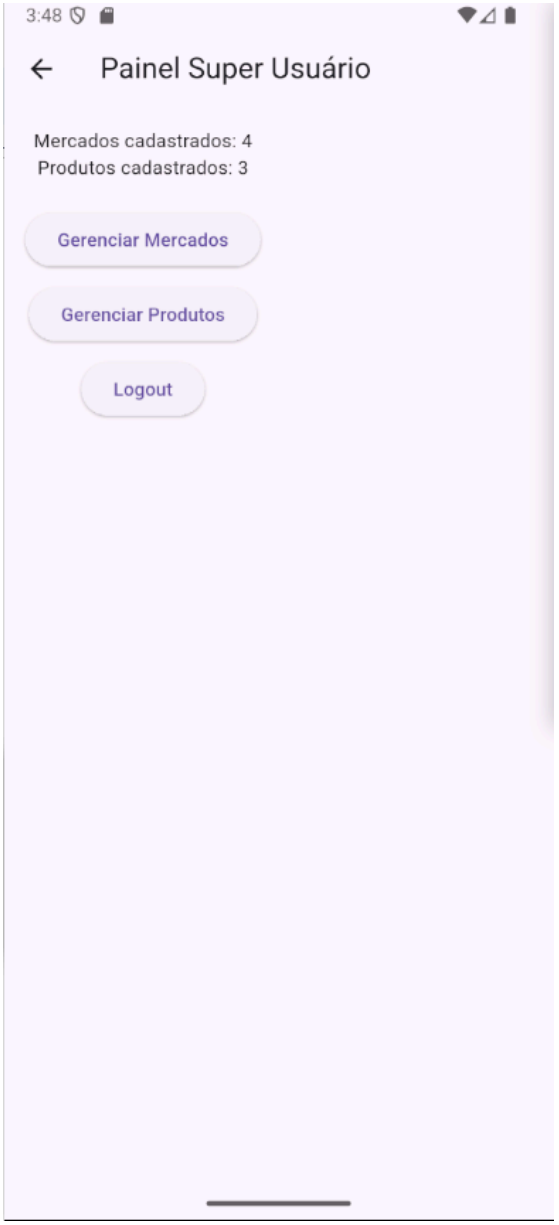
49. Requisitos + ambiguidades + modelo de dados
50. Ambiente (Flutter + VSCode + Supabase)
51. Backend (SQL, policies, superusuário, storage)
52. Estrutura do app Flutter
53. Funcionalidades previstas no documento
54. Autenticação, permissões e superusuário
55. Checklist final (cada RF → ✓ ou ✗)
56. PROJETO CONCLUÍDO

FORMATO DE ENTREGA

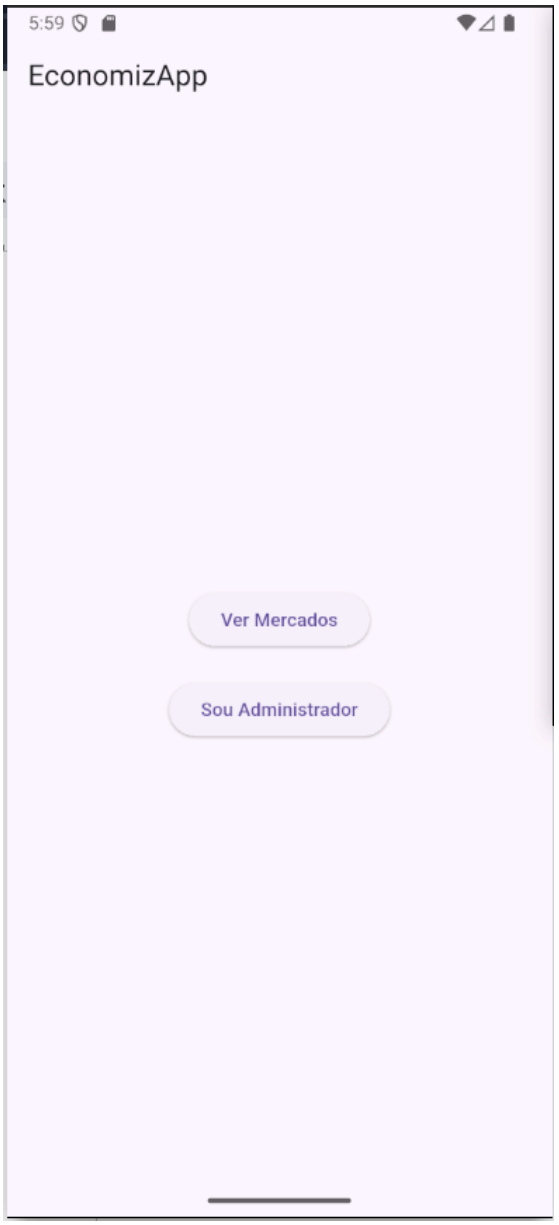
- Código completo, atual e funcional, organizado por arquivos
- Comandos de terminal quando necessários
- Sem adicionar nada além do que está no documento

CHATGPT

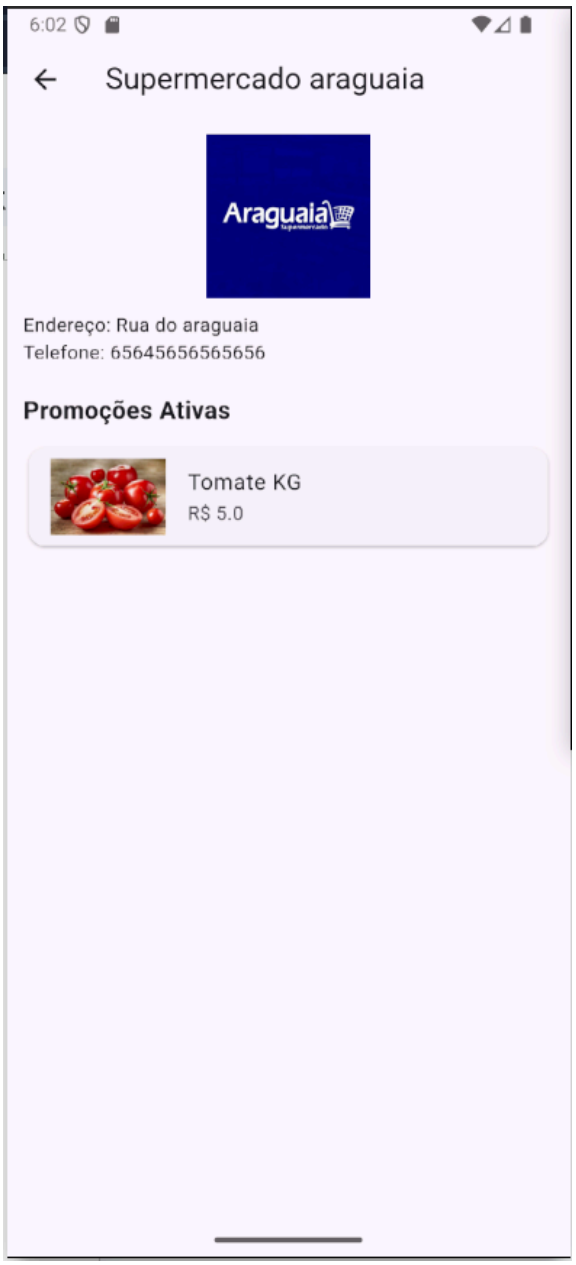
Com o ChatGPT foram feitas 23 intervenções para correções de erros que estão relacionados a códigos no Flutter e no supabase. O MVP final atendeu aos requisitos.



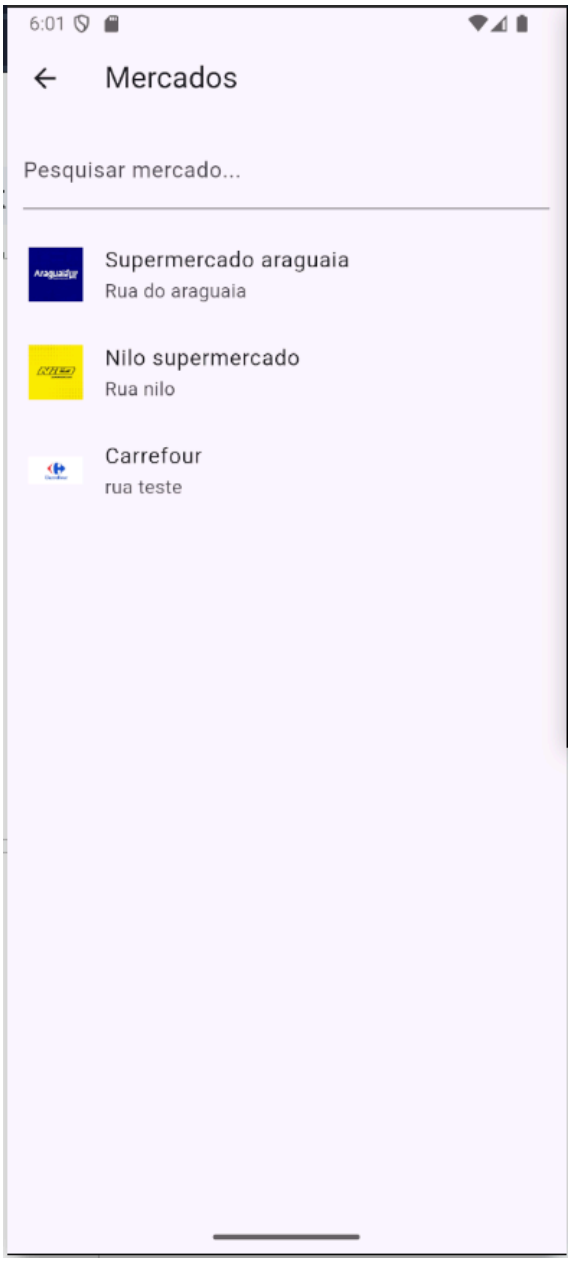
Tela do super usuário



Tela inicial



Tela de detalhes do mercado

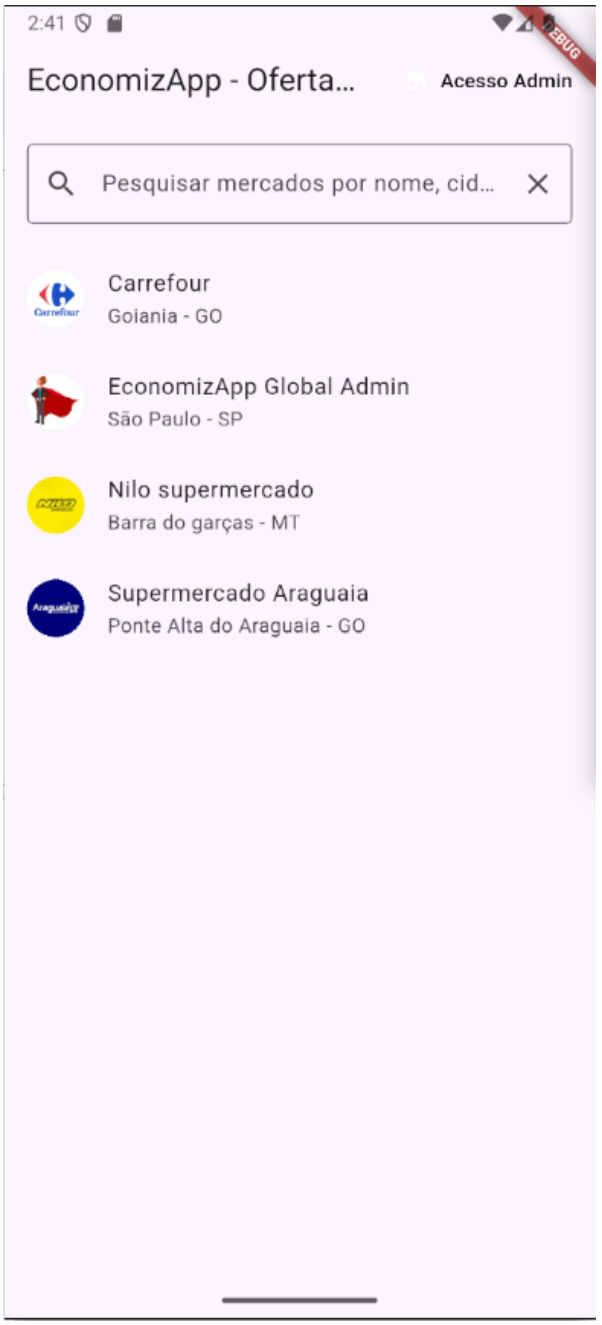


Tela de mercados e pesquisa

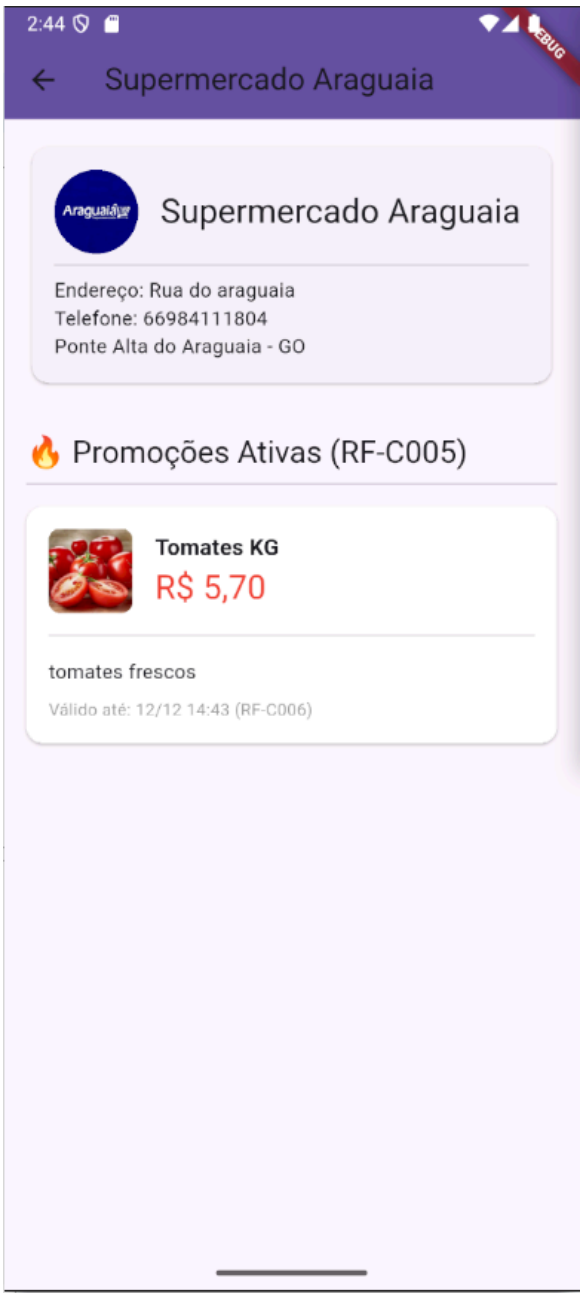
GEMINI

Com o Gemini foram feitas 14 interações, todas as funcionalidades testadas estão funcionando normalmente. Na aplicação MVP o super usuário foi tratado como um mercado, podendo adicionar produtos.

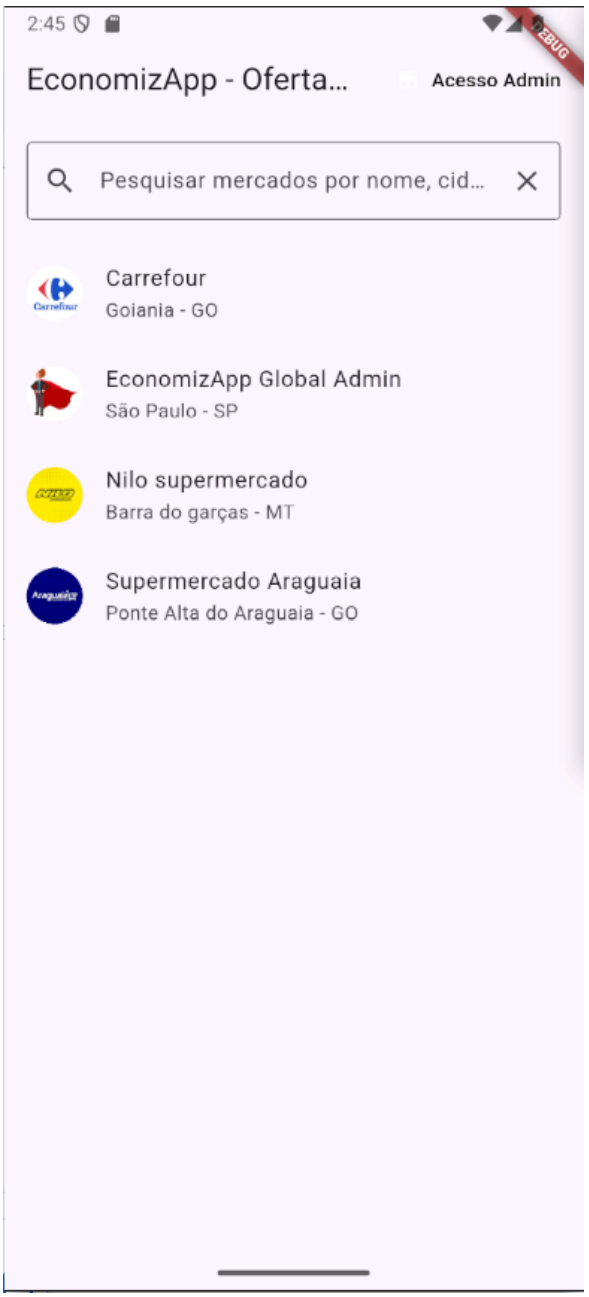
Tela do super usuário



Tela inicial



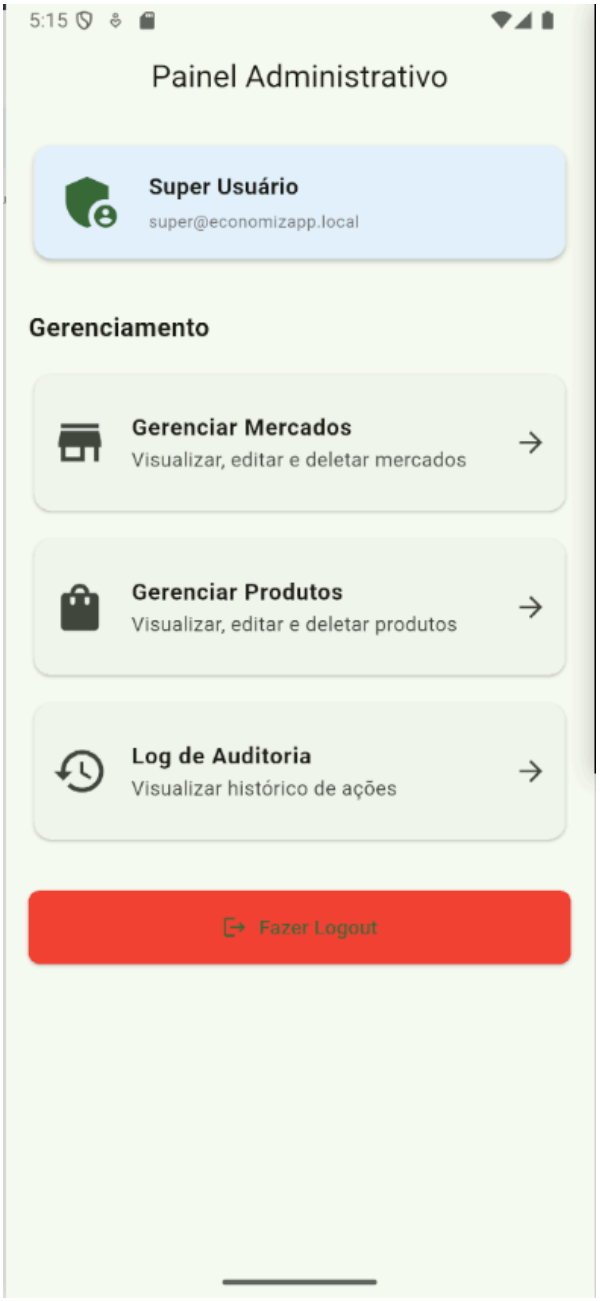
Tela de detalhes do mercado



Tela de mercados e pesquisa

CLAUDE

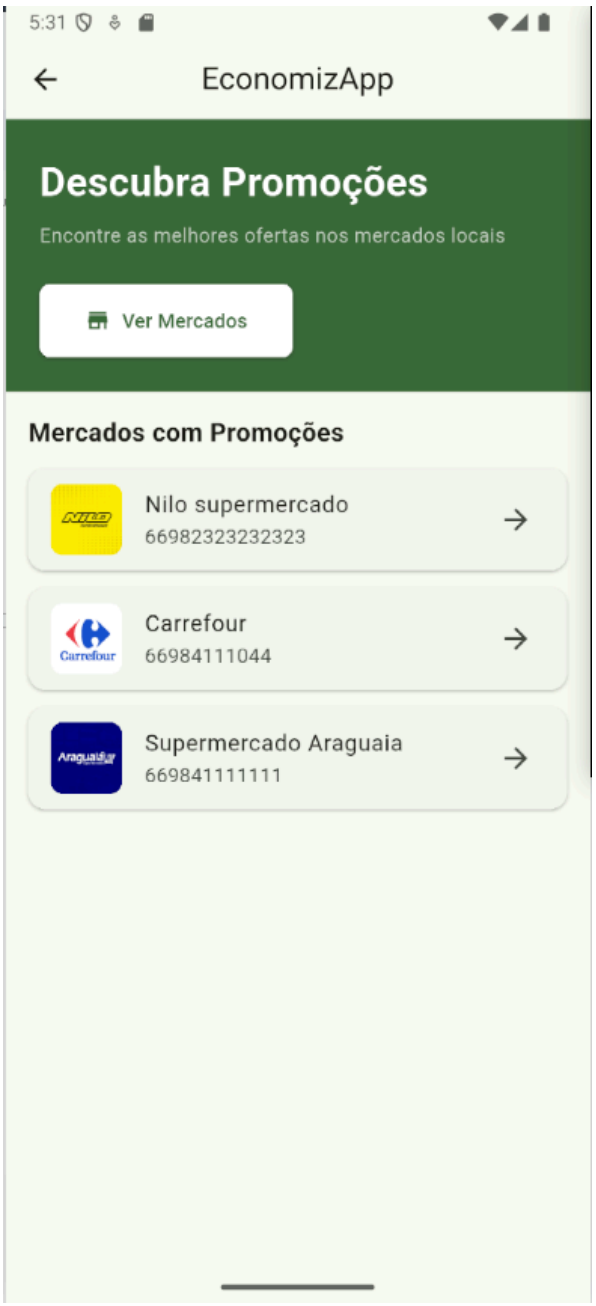
O Claude teve 23 interações, não conseguiu desenvolver o fluxo de esquecer a senha completamente, o link de recuperação enviado não redireciona para a aplicação. A última intervenção foi pedido para que desenvolvesse uma solução definitiva e a resposta foi a remoção do recurso de recuperação de senha, por isso, não foi possível finalizar este recurso. Todos os modelos disponíveis do Claude são gratuitos mas possuem um limite de uso, os testes foram feitos com o Haiku 4.5 a versão de respostas rápidas.



Tela do super usuário

Tela inicial

Tela de detalhes do mercado



Tela de mercados e pesquisa

Requisitos Funcionais

Código	Requisito Funcional (RF)	Gemini	ChatGPT	Claude
RF-G	**(Geral)**			
RF-G001	O sistema deve possuir duas interfaces (ou modos) distintas: uma para o Cliente Administrador e outra para o Cliente Comum.	✅	✅	✅
RF-A	**(Módulo Administrador - Mercado)**			
RF-A001	O admin deve poder realizar um cadastro de conta de mercado, fornecendo: foto (logo), e-mail, senha, endereço completo e telefone.	✅	✅	✅
RF-A002	O sistema deve validar o e-mail (formato e unicidade) durante o cadastro.	✅	✅	✅
RF-A003	O admin deve poder realizar login usando e-mail e senha.	✅	✅	✅

RF-A004	(Implícito) O admin deve poder realizar a recuperação de senha (ex: "Esqueci minha senha").	✅	✅	❌ não completou o fluxo de recuperação de senha
RF-A005	O admin deve poder editar as informações de seu perfil (foto, endereço, telefone).	✅	✅	✅
RF-A006	O admin (logado) deve poder cadastrar um novo produto em promoção, informando: foto do produto, preço promocional, tempo de promoção (data/hora de início e fim) e descrição.	✅	✅	✅
RF-A007	O admin (logado) deve poder editar as informações de um produto em promoção cadastrado.	✅	❌ A edição da hora e data não está funcionando.	✅
RF-A008	O admin (logado) deve poder remover um produto em promoção.	✅	✅	✅
RF-A009	O admin (logado) deve poder visualizar uma lista de todos os seus produtos em promoção (ativos e talvez inativos).	✅	✅	✅

RF-C	**(Módulo Cliente Comum - Usuário)**			
RF-C001	O usuário deve poder acessar o aplicativo sem a necessidade de cadastro ou login.	✅	✅	✅
RF-C002	O usuário deve poder visualizar uma lista de mercados cadastrados.	✅	✅	✅
RF-C003	O usuário deve poder realizar pesquisas de mercados (ex: por nome).	✅	✅	✅
RF-C004	O usuário deve poder visualizar a página de perfil de um mercado (com suas informações: foto, endereço, telefone).	✅	✅	✅
RF-C005	O usuário, ao acessar a página de um mercado, deve poder visualizar a lista de produtos em promoção ativos daquele mercado.	✅	✅	✅
RF-C006	O usuário deve poder visualizar os detalhes de um produto em promoção (foto, preço, descrição, validade da promoção).	✅	✅	✅

Requisitos não funcionais

ID	Requisito Não Funcional (RNF)	Gemini	ChatGPT	Claude
RNF001	A aplicação deve ser desenvolvida em Flutter, visando compatibilidade somente com Android.	✅	✅	✅
RNF002	O carregamento das listas (mercados e produtos) deve ser otimizado.	✅	✅	✅
RNF003	A interface deve ser intuitiva e limpa.	✅	✅	✅
RNF004	As senhas dos administradores devem ser armazenadas no banco de dados.	✅	✅	✅
RNF005	Toda a comunicação entre o aplicativo e o backend deve ser realizada via protocolo seguro.	✅	✅	✅
RNF006	As fotos (logos, produtos) devem ser armazenadas em um serviço de storage gratuito (Supabase) e não diretamente no banco de dados.	✅	✅	✅
RNF007	O sistema deve gerenciar o estado da promoção: produtos com "tempo de promoção" expirado não devem ser exibidos para o Cliente Comum.	✅	✅	✅

Observações

Durante o processo de correção de erros nas IAs ambas tiveram alguns problemas em comum:

1 - O erro indica um problema de recursão infinita na política RLS do Supabase. O problema está na policy que permite o admin criar sua própria conta.

O problema de recursão infinita na política RLS (Row Level Security) do Supabase ocorre quando uma política para uma tabela tenta, direta ou indiretamente, consultar a própria tabela ou uma tabela relacionada que, por sua vez, consulta a original, para determinar o acesso, criando um loop.

2 - Dificuldades na criação dos recursos do super usuário, como exclusão e edição dos mercados e seus produtos, mas todas as 3 IAs conseguiram resolver os problemas deste recurso.

3 - Dificuldades na criação do recurso de recuperação de senha. Apenas o Claude não conseguiu desenvolver este recurso.

Detalhes das intervenções

ChatGPT

1ª intervenção:

Erro no código SQL no Supabase:

2ª intervenção:

Erro na solução no código SQL no Supabase:

3ª intervenção:

Erro no código Flutter:

4ª intervenção:

Erro no código Flutter:

5ª intervenção:
Erro no código Flutter:

6ª intervenção:

Erro no código Flutter:

7ª intervenção:

Erro no código Flutter:

8ª intervenção:

Erro no código Flutter:

9ª intervenção:
Erro nas policies do Supabase:

10ª intervenção

Erro de requisitos da documentação (Fluxo do super usuário faltando):

11ª intervenção
Geração dos códigos SQL para o super usuário:

12ª intervenção:
Ausência de recurso no Flutter (Fluxo esqueceu a senha):

13ª intervenção:
Erro de configuração do Supabase para o Fluxo de redefinição de senha:

14ª intervenção:
Erro de código no Flutter para o fluxo de redefinição de senha:

15ª intervenção:
Teste do super usuário e não atendeu às especificações da documentação

16ª intervenção:
Configuração no Supabase para o super usuário:

17ª intervenção
Modificação do fluxo de super usuário no Flutter:

18ª intervenção
Erro no código Dart no fluxo de super usuário.

19ª intervenção
Erro no código Dar no fluxo de redefinição de senha.

20ª intervenção
Ausência de recursos (cadastro com hora de início e término da promoção).

21ª intervenção
Erro de edição nos dados de data e hora da promoção.

22ª intervenção
Erro de política de segurança no Supabase (data de início no futuro da promoção).

23ª intervenção
Última intervenção com problema de não atualização da edição de data e hora de promoção, o resultado foi a não conclusão deste deste recurso.

Gemini
1ª intervenção:
Erro de política de segurança RLS do Supabase.

2ª intervenção:
Erro de código Dart no Flutter.

3º intervenção:
Erro da política de segurança RLS do Supabase.

4ª intervenção:
Erro de requisitos da documentação (Fluxo do super usuário faltando):

5ª intervenção:
Erro no SQL do supabase.

6ª intervenção:
Erro de código Dart no Flutter.

7ª intervenção:
Erro de código Dart no Flutter.

8ª intervenção:
Erro de configuração no Flutter.

9ª intervenção:
Ausência do fluxo de redefinição de senha.

10ª intervenção:
Erro de código Dart no Flutter.

11ª intervenção:
Erro de configuração no Flutter.

12ª intervenção:
Erro de configuração no Flutter e no Supabase.

13ª intervenção:
Erro da política de segurança RLS do Supabase.

14ª intervenção:
Erro de código Dart no Flutter.
Esta foi a última intervenção com a correção do recurso de pesquisa de mercados.

Claude
1ª intervenção:
Erro no código Flutter.

2ª intervenção:
Erro no código Flutter.

3ª intervenção:
Erro no código Flutter.

4ª intervenção:
Erro de políticas de segurança (RLS) no supabase.

5ª intervenção:
Erro de políticas de segurança (RLS) no supabase.

6ª intervenção:
Erro de políticas de segurança (RLS) no supabase.

7ª intervenção:
Erro de políticas de segurança (RLS) no supabase.

8ª intervenção:
Erro de políticas de segurança (RLS) no supabase.

9ª intervenção:
Erro de políticas de segurança (RLS) no supabase.

10ª intervenção:
Erro de políticas de segurança (RLS) no supabase.

11ª intervenção:
Erro de políticas de segurança (RLS) no supabase.

12ª intervenção:
Erro de políticas de segurança (RLS) e SQL no supabase.

13ª intervenção:
Ausência do fluxo de redefinição de senha (configuração no supabase e Flutter)

14ª intervenção:
Erro de configuração no supabase

15ª intervenção:
Erro de configuração no Flutter e supabase

16ª intervenção:
Erro de configuração no Flutter e supabase

17ª intervenção:
Erro de configuração no Flutter e supabase

18ª intervenção:
Erro de configuração no Flutter e supabase

19ª intervenção:
Erro no código Flutter

20ª intervenção:
Erro no código Flutter

21ª intervenção:
Erro no código Flutter

22ª intervenção:
Erro no código Flutter

23ª intervenção:

Erro no código Flutter

A última intervenção envolveu a correção para o fluxo de redefinição de senha, o resultado foi a não conclusão deste fluxo.

1. Quadro Resumo Comparativo

Métrica	ChatGPT	Gemini	Claude
Total de Intervenções	23	14	23
Maior Foco de Erro	Frontend (Flutter)	Misto (Equilibrado)	Backend (Supabase/RLS)
Resultado Final	❌ Não concluiu (Promoções)	✅ Concluiu (Pesquisa)	❌ Não concluiu (Senha)

2. Classificação por Domínio Tecnológico

Backend (Supabase & SQL)

Foco: Banco de dados, Autenticação e Segurança.

- **ChatGPT:** 5 intervenções (SQL, Policies, Configuração).
- **Gemini:** 6 intervenções (RLS predominante, SQL, Configuração).
- **Claude: 13 intervenções** (Forte dificuldade com RLS - Row Level Security e Configurações mistas).

Frontend (Flutter & Dart)

Foco: UI, Lógica do cliente e Sintaxe.

- **ChatGPT: 13 intervenções** (Muitos erros de código e configuração).
- **Gemini:** 6 intervenções (Erros de código Dart e configuração).
- **Claude:** 9 intervenções (Erros de código concentrados no final).

Requisitos & Documentação

Foco: Entendimento do escopo e "Features" faltantes.

- **ChatGPT:** 5 intervenções (Esquecimento de fluxos inteiros: Super Usuário, Senha, Promoção).
- **Gemini:** 2 intervenções (Super usuário e fluxo de senha).
- **Claude:** 1 intervenção (Ausência do fluxo de senha).

Modelo IA	Frontend (Flutter/Dart)	Backend (Supabase/SQL/RLS)	Requisitos(Docs/Features)	Integração (Misto/Config)	Total
ChatGPT	12	7	4	0	23
Gemini	7	4	2	1	14
Claude	8	10	1	4	23

Descrição das Colunas (Taxonomia do Estudo)

1. Modelo IA

Identifica o Grande Modelo de Linguagem (LLM) submetido ao teste. Nesta análise, representa a **variável independente** do estudo de caso (ChatGPT, Gemini e Claude), permitindo a comparação direta de desempenho entre diferentes arquiteturas e treinamentos.

2. Frontend (Flutter/Dart)

Refere-se exclusivamente à camada **Client-side**.

- **O que abrange:** Erros de sintaxe na linguagem Dart, construção incorreta da árvore de Widgets do Flutter, falhas na gestão de estado (Providers/Bloc) e lógica de apresentação.
- **Significado no Contexto:** Uma contagem alta aqui indica que o modelo tem dificuldade em gerar código visualmente funcional ou que "alucina" métodos e propriedades inexistentes no framework Flutter.

3. Backend (Supabase/SQL/RLS)

Refere-se à camada **Server-side** e de **Persistência de Dados**.

- **O que abrange:** Criação de tabelas, *Stored Procedures* em PL/pgSQL e, criticamente, as **Policies RLS (Row Level Security)**.
- **Significado no Contexto:** Indica a competência do modelo em lidar com lógica de banco de dados e regras de segurança. No caso do Claude, a alta incidência aqui revelou uma falha grave em compreender como o Supabase gerencia permissões de acesso.

4. Requisitos (Docs/Features)

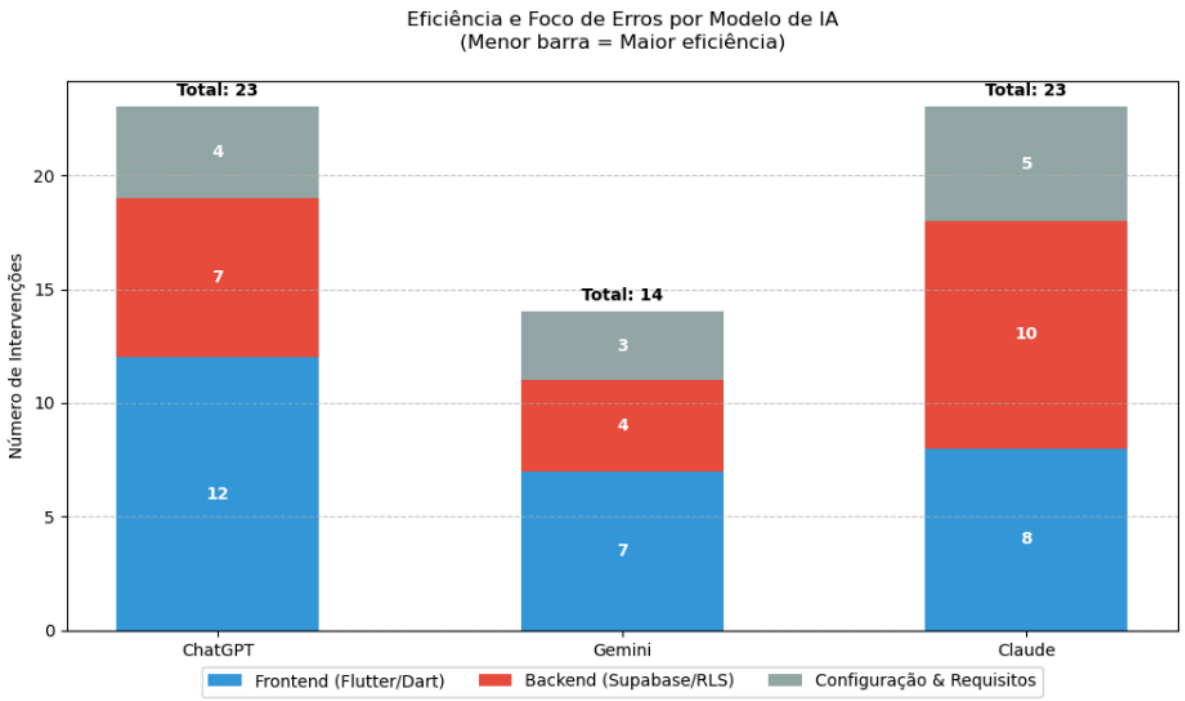
Refere-se à **Engenharia de Requisitos** e à capacidade de **Retenção de Contexto**.

- **O que abrange:** Situações onde o código estava correto, mas a funcionalidade não atendia ao que foi pedido (ex: esquecer o fluxo de "Esqueci minha senha" ou ignorar regras de negócio documentadas).
- **Significado no Contexto:** Mede a "atenção" do modelo. Uma pontuação alta aqui (como no ChatGPT) sugere que o modelo tem uma "janela de contexto" volátil ou falha em processar instruções longas e complexas.

5. Integração (Misto/Config)

Refere-se à **Configuração de Ambiente** e à comunicação entre as partes (Middleware).

- **O que abrange:** Erros onde o Frontend não consegue falar com o Backend devido a credenciais erradas, chaves de API mal configuradas, ou dependências (pubspec.yaml) conflitantes.
- **Significado no Contexto:** Representa a dificuldade "sistêmica". Ocorre quando o modelo gera as partes isoladamente corretas, mas falha em fazê-las funcionar como um sistema unificado.


Eficiência e Foco de Erros por Modelo de IA
(Menor barra = Maior eficiência)
Total: 23
Total: 14
Total: 23
4
7
12
3
4
7
5
10
8
Número de Intervenções
0
5
10
15
20
ChatGPT
Gemini
Claude
Frontend (Flutter/Dart)
Backend (Supabase/RLS)
Configuração & Requisitos